UNIVERSIDADE FEDERAL DO CEARÁ

Test Smells em projetos OpenSource

Disciplina: Qualidade de Software
Equipe: Francisco Henrique Nunes da Silva (566487) e Tobias Alves Couras (565022)
Projeto: Projeto Jarvis Original
Fork: Projeto Jarvis Fork

Refatoração 1

branch: refactor/conditional-test-logic
commit (94149ba): remove diagnostic configuration loop

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Conditional Test Logic **Arquivo editado:** tests/test_mcp_e2e.py **Linha:** 38 (função test_configuration_loading)
Minhas principais dificuldades na remoção do code smell são:
Não tive dificuldade. O for só percorria os servidores MCP para imprimir o nome e o comando no terminal, ou seja, não validava nada. Bastou conferir que o assert logo abaixo (isinstance(mcps, dict)) continuava sozinho garantindo o objetivo do teste.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Conditional Test Logic e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Dead Code / Remove Diagnostic Code: apaguei o laço e os prints que só serviam de depuração. – Teste continua com o assert original, então o comportamento verificado não mudou.

Refatoração 2

branch: refactor/conditional-test-logic
commit (9663a21): remove diagnostic discovery loop

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Conditional Test Logic **Arquivo editado:** tests/test_mcp_e2e.py **Linha:** 92 (função test_mcp_discovery_with_mock)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. Mesmo caso do commit anterior: um for que apenas imprimia o nome de cada ferramenta descoberta. A verificação real já era feita pelo assert que compara actual_tools com expected_tools.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Conditional Test Logic e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Dead Code / Remove Diagnostic Code: removi o laço de impressão. – Mantive o assert de comparação entre ferramentas esperadas e encontradas.

Refatoração 3

branch: refactor/conditional-test-logic
commit (dc71d0c): assert every tool description without a branch

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Conditional Test Logic **Arquivo editado:** tests/test_mcp_e2e.py **Linha:** 138 (função test_tool_registration_in_descriptions)
Minhas principais dificuldades na remoção do code smell são:
Tive uma pequena dificuldade: o teste usava um if dentro do for e uma variável success só para acumular o resultado. Troquei o if por um assert direto, mas o laço continuou, então esse commit resolveu só metade do problema e precisei de outro commit depois para tirar o for.

Usei o CATSPY para:
Usei o CATSPY para conferir novamente a definição do smell e a sugestão de refatoração para laços sem papel de verificação, que é remover a lógica condicional do corpo do teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Dead Code / Remove Diagnostic Code: removi a condição de impressão. – Mantive o assert de comparação entre ferramentas esperadas e encontradas.

Refatoração 4

branch: refactor/conditional-test-logic
commit (0ac01fb): unroll tool-name checks

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Conditional Test Logic **Arquivo editado:** tests/test_mcp_e2e.py **Linha:** 137 (função test_tool_registration_in_descriptions)
Minhas principais dificuldades na remoção do code smell são:
Aqui a dúvida foi de decisão: ao desenrolar o laço eu passo a escrever os nomes das ferramentas direto no teste (server1__tool1 e server2__tool2), perdendo um pouco de generalidade. Como são só duas ferramentas fixas, achei o trade-off aceitável pela clareza que o teste ganha.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Conditional Test Logic e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Unroll Loop: um assert por ferramenta, sem iteração. – Teste passa a ser linear e cada falha aponta exatamente qual nome faltou.



UNIVERSIDADE FEDERAL DO CEARÁ

Refatoração 5

branch: refactor/conditional-test-logic
commit (bcc242e): make naming test double unconditional

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Conditional Test Logic **Arquivo editado:** tests/test_mcp_e2e.py **Linha:** 163 (FakeMCPClient.list_tools em test_tool_name_format)
Minhas principais dificuldades na remoção do code smell são:
Foi simples. O if server_name == 'my-server' dentro do dublê (fake) só existia para devolver lista vazia para outros servidores, mas o teste só usa um servidor. Só precisei checar que nenhum outro servidor era consultado antes de remover o desvio.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Conditional Test Logic e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Conditional / Simplify Test Double: o fake sempre devolve a mesma lista, sem ramificação. – Eliminação do ramo morto (return []).

Refatoração 6

branch: refactor/conditional-test-logic
commit (06bd431): remove diagnostic naming loop

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Conditional Test Logic **Arquivo editado:** tests/test_mcp_e2e.py **Linha:** 186 (função test_tool_name_format)

Minhas principais dificuldades na remoção do code smell são:
Não houve dificuldade. Era outro for só de impressão, depois do assert que já compara actual_names com expected_names.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Conditional Test Logic e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Dead Code / Remove Diagnostic Code: retirei o laço e os prints.

Refatoração 7

branch: refactor/conditional-test-logic
commit (60ea9d6): supply server outcomes without branches

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Conditional Test Logic **Arquivo editado:** tests/test_mcp_e2e.py **Linha:** 206 (FaultyMCPClient.list_tools em test_error_handling)
Minhas principais dificuldades na remoção do code smell são:
Essa foi a mais trabalhosa. O dublê FaultyMCPClient tinha if/elif para decidir qual servidor funciona, qual lança erro e qual retorna vazio. Troquei por um Mock com side_effect em sequência, mas isso faz o resultado depender da ordem em que os servidores são consultados (good, bad, empty), então tive que garantir que a ordem do dicionário de configuração batia com a lista. Também troquei Exception genérica por ConnectionError.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Conditional Test Logic e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Replace Hand-written Fake with Mock: o Mock com side_effect substitui a classe com if/elif. – Replace Conditional with Declarative Setup: as respostas por servidor viram uma lista ordenada. – Troca de Exception por ConnectionError para deixar o cenário mais específico.

Refatoração 8

branch: refactor/conditional-test-logic
commit (f60e97c): assert healthy and failed server results directly

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Conditional Test Logic **Arquivo editado:** tests/test_mcp_e2e.py **Linha:** 222 (função test_error_handling)
Minhas principais dificuldades na remoção do code smell são:
Essa foi a mais trabalhosa o if/else só imprimia ✅ ou ❌ e chamava assert False no caso de erro. Para trocar por asserts diretos precisei descobrir o formato real de mcp_errors e só então pude também verificar o erro do servidor ruim, o que deixou o teste mais forte do que era.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Conditional Test Logic e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Replace Conditional with Assertion: o if/else virou dois asserts diretos. – Strengthen Assertion: passei a verificar também o conteúdo de mcp_errors.

Refatoração 9

branch: refactor/sleepy-test
commit (bc8dd2a): check empty chat rejection synchronously

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_chat_submission.py **Linha:** 281 (time.sleep(0.3) em test_empty_or_whitespace_does_not_run_engine)
Minhas principais dificuldades na remoção do code smell são:
Não tive dificuldade. A consulta vazia é rejeitada de forma síncrona dentro de submit_text_query, então não existe nada em segundo plano para esperar. Bastou apagar o sleep e conferir que o teste continuava passando.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Unnecessary Sleep: apaguei o time.sleep(0.3), pois o comportamento é síncrono.

Refatoração 10

branch: refactor/sleepy-test
commit (ba28211): coordinate chat cancellation with events

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_chat_submission.py **Linha:** 293 (time.sleep(0.3) em test_cancel_drops_reply_and_emits_complete_none)
Minhas principais dificuldades na remoção do code smell são:
Foi a parte mais chata dos sleeps. O sleep fazia o motor 'demorar' para dar tempo do cancelamento acontecer no meio. Troquei por um threading.Event (release_engine) que só é liberado depois do cancel, e usei assert release_engine.wait(timeout=2) para o teste não travar para sempre se algo falhar. Precisei pensar bem na ordem: started, cancel,

release.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Replace Sleep with Synchronization Primitive: threading.Event no lugar do time.sleep. – Uso de timeout nos wait() para evitar bloqueio infinito.

Refatoração 11

branch: refactor/sleepy-test
commit (d46cee0): wait for first chat worker before busy check

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_chat_submission.py **Linha:** 387 (time.sleep(0.1) em test_second_submission_fires_on_busy; espera relacionada na linha 376)
Minhas principais dificuldades na remoção do code smell são:
Dificuldade média o sleep(0.1) servia para 'dar um tempinho' para a primeira thread pegar o lock, o que é uma corrida de condição disfarçada. Criei o evento first_started, disparado dentro do slow_engine, e o teste só envia a segunda submissão depois dele. Também transformei o slow_done.wait em assert para falhar caso estoure o tempo.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Replace Sleep with Synchronization Primitive: evento first_started sinaliza que a thread começou. – Add Assertion on Wait: assert nos wait() para detectar timeouts.



UNIVERSIDADE
FEDERAL DO CEARÁ

Refatoração 12

branch: refactor/sleepy-test
commit (2c3f910): advance chat memory clock without sleeping

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_chat_submission.py **Linha:** 615 (_time.sleep(1.2) em test_empty_when_hot_window_has_aged_out)
Minhas principais dificuldades na remoção do code smell são:
Dificuldade média o teste esperava 1,2 s reais para a janela quente 'envelhecer'. Para tirar o sleep tive que descobrir que o módulo jarvis.memory.conversation usa time.time() e fazer um monkeypatch no atributo time do módulo com um relógio falso controlável (clock[0] += ...). Precisei ter cuidado para substituir só no módulo certo.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Replace Real Clock with Fake Clock: monkeypatch de conversation.time por um relógio controlável. – Remove Sleep: o tempo avança alterando a variável, sem esperar.

Refatoração 13

branch: refactor/sleepy-test
commit (8ac1090): advance collection timeout clock directly

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_state_manager.py

Linha: 95 (time.sleep(0.06) em test_silence_timeout_triggers_collection_complete)
Minhas principais dificuldades na remoção do code smell são:
Foi parecido com o commit anterior, então já sabia o caminho: relógio falso via monkeypatch no módulo state_manager. A única atenção foi manter o mesmo valor de timeout (0.05) e avançar o relógio o suficiente (voice_collect_seconds + 0.01) para o timeout disparar. Deixei os outros sleeps do arquivo para outra rodada.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Replace Real Clock with Fake Clock: monkeypatch de state_manager.time. – Remove Sleep: o relógio é avançado diretamente no teste.

Refatoração 14

branch: refactor/unknown-test
commit (f903d60): assert callback-free chat submission completes

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Unknown Test **Arquivo editado:** tests/test_chat_submission.py **Linha:** 210 (test_callbacks_are_optional)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade o teste não tinha nenhum assert, só um comentário dizendo que chegar ao fim já provava o funcionamento, e ainda dependia de time.sleep(0.5). Tive que pensar no que realmente deveria ser verificado: que o motor foi executado mesmo sem callbacks. Resolvi com um Event disparado dentro do motor falso e um assert no wait.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Unknown Test e ver uma possível refatoração

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Add Assertion: assert engine_finished.wait(timeout=2). – Replace Sleep with Synchronization Primitive: o sleep(0.5) foi trocado por Event.

Refatoração 15

branch: refactor/unknown-test
commit (3674982): assert unbooted chat session stays empty

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Unknown Test **Arquivo editado:** tests/test_chat_submission.py **Linha:** 648 (test_new_chat_session_noops_when_daemon_not_booted)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade o teste só chamava new_chat_session() com o comentário 'must not raise'. Tive que ler a função no daemon para descobrir o que ela retorna quando o daemon não foi inicializado, e vi que devolve False, então esse retorno virou a asserção.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Unknown Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Add Assertion: assert daemon.new_chat_session() is False. – Replace Implicit Check (no exception) with Explicit Assertion.

Refatoração 16

branch: refactor/unknown-test
commit (218d93b): verify removed mutation listener stays inactive

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Unknown Test **Arquivo editado:** tests/test_graph_mutation_listener.py **Linha:** 111 (test_unregister_is_idempotent)
Minhas principais dificuldades na remoção do code smell são:
Foi o que mais exigiu raciocínio entre os Unknown Tests. O teste só chamava unregister duas vezes e esperava não lançar erro. Para ter uma asserção com sentido, fiz o callback registrar chamadas numa lista, criei um nó no graph_store depois de remover o listener e verifiquei que a lista continua vazia. Precisei incluir a fixture graph_store no teste.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Unknown Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Add Assertion: assert events == []. – Introduce Test Spy: o callback grava os eventos recebidos. – Add Fixture Parameter: uso da fixture graph_store para disparar uma mutação real.

Refatoração 17

branch: refactor/unknown-test
commit (bc6f9cd): assert stopped tune remains inactive

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Unknown Test **Arquivo editado:** tests/test_tune_player.py **Linha:** 135 (test_stop_is_idempotent)
Minhas principais dificuldades na remoção do code smell são:
Não houve dificuldade. Bastou olhar que o TunePlayer tem o método is_playing() e adicionar um assert not tp.is_playing() depois das duas chamadas de stop_tune().

Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Unknown Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Add Assertion: assert not tp.is_playing().

Refatoração 18

branch: refactor/unknown-test
commit (328d2e4): assert absent crash marker remains absent

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Unknown Test **Arquivo editado:** tests/test_desktop_app.py **Linha:** 497 (chamada de mark_session_clean_exit em teste de marcador inexistente)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O teste apagava o marcador de crash, chamava mark_session_clean_exit() com o comentário 'Should not raise' e terminava. Adicionei assert not crash_marker.exists(), que é o estado final esperado.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Unknown Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Add Assertion: assert not crash_marker.exists().

Refatoração 19

branch: refactor/verbose-test
commit (055134f): focus MCP discovery test on observable results

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Verbose Test **Arquivo editado:** tests/test_mcp_e2e.py **Linha:** 52 (test_mcp_discovery_with_mock, linhas 52 a 100)
Minhas principais dificuldades na remoção do code smell são:
Dificuldade média. O teste tinha quase 50 linhas por causa de prints, try/except com traceback, configuração duplicada e um import de Mock sem uso. A parte difícil foi decidir o que era ruído e o que era verificação de verdade. Removi o try/except que escondia o erro real e ainda aproveitei para verificar errors == {} e a descrição de uma ferramenta, mantendo o foco no resultado.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Verbose Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Irrelevant Code: tirei prints, traceback e import sem uso. – Remove Try/Except Wrapper: deixei a falha aparecer naturalmente no pytest. – Inline Variable: a configuração fake foi passada direto à chamada. – Focus Assertions: asserts diretos sobre o resultado.

Refatoração 20

branch: refactor/verbose-test
commit (d1e7dab): focus MCP description test on tool content

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Verbose Test **Arquivo editado:** tests/test_mcp_e2e.py **Linha:** 103 (test_tool_registration_in_descriptions, linhas 103 a 149)
Minhas principais dificuldades na remoção do code smell são:

Dificuldade média. O teste misturava prints, try/except, o laço com flag de sucesso e um dicionário de ToolSpec com indentação espalhada em muitas linhas. Reduzi o dicionário, passei list(mcp_tools) como ferramentas permitidas e troquei a verificação por quatro asserts claros (nome e descrição de cada ferramenta). Fiquei em dúvida se removia os prints de depuração, mas como o pytest já mostra o erro, mantive só o necessário.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Verbose Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Irrelevant Code: prints e mensagens de depuração eliminados. – Remove Try/Except Wrapper. – Simplify Test Data: dicionário de ToolSpec compactado e reaproveitado. – Replace Conditional/Flag with Assertions: verificação direta de nome e descrição.

Refatoração 21

branch: refactor/sleepy-test
commit (f06868a): refactor: advance max collection clock directly

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_state_manager.py **Linha:** 105 (test_max_duration_timeout)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O teste fazia três time.sleep(0.02) dentro de um loop, adicionando texto à coleta entre cada pausa para não cair no timeout de silêncio, até passar dos 50ms do limite máximo. Precisei observar que o check_collection_timeout compara time.time() tanto com o início da coleta quanto com a última fala, então o relógio falso tinha que avançar menos que o timeout de silêncio entre um add_to_collection e outro, para o teste continuar validando o limite máximo e não o de silêncio.

Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Unnecessary Sleep: troquei os time.sleep por um relógio controlado via monkeypatch no time do state_manager, avançando o tempo de forma determinística. – Unroll Loop / Add Assertion: removi o loop e adicionei um assert de que a coleta ainda não expira logo antes do limite, além do assert final de que expira depois dele.

Refatoração 22

branch: refactor/sleepy-test
commit (e879e68): refactor: fire hot window activation timer directly

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_state_manager.py **Linha:** 126 (test_schedule_hot_window_activation)
Minhas principais dificuldades na remoção do code smell são:
Dificuldade média. O teste agendava a ativação da hot window e dava um time.sleep(0.1) esperando o threading.Timer real disparar depois dos 50ms de echo tolerance. Aqui o relógio falso não resolvia, porque a ativação depende de uma thread de timer e não de time.time(). Precisei observar que o callback activate confere se o timer que disparou ainda é o timer registrado no StateManager, então o timer falso tinha que ser o mesmo objeto guardado em _hot_window_activation_timer para a ativação não ser ignorada.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Unnecessary Sleep: substituí o threading. Timer por um timer falso via monkeypatch e disparei o callback manualmente, sem esperar tempo real.

– Add Assertion: adicionei um assert de que o timer foi agendado com o atraso de echo_tolerance, mantendo os asserts de inativa antes e ativa depois do disparo.

Refatoração 23

branch: refactor/sleepy-test
commit (2b3a139): refactor: cancel hot window activation without sleeping

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_state_manager.py **Linha:** 139 e 143 (test_cancel_hot_window_activation)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O teste dava um time.sleep(0.02) antes de cancelar a ativação e depois um time.sleep(0.15) para passar do tempo em que o timer real teria disparado, só então conferindo que a hot window continuava inativa. Precisei observar que, sem o timer real, eu tinha que simular o pior caso: o callback do timer cancelado ainda rodar atrasado. Como o _activate ignora timers que não são mais o registrado, chamar o callback depois do cancelamento comprova que o cancelamento funciona.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Unnecessary Sleep: removi os dois time.sleep e substituí o threading.Timer por um timer falso via monkeypatch, disparando o callback manualmente depois do cancelamento. – Add Assertion: adicionei um assert de que o timer foi de fato cancelado, mantendo o assert de que a hot window não ativa.

Refatoração 24

branch: refactor/sleepy-test
commit (eae9130): refactor: fire activation timer during collection directly

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_state_manager.py **Linha:** 156 e 160 (test_hot_window_not_activated_during_collection)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O teste agendava a ativação da hot window, dormia 20ms, iniciava uma coleta e dormia mais 100ms para passar do tempo em que o timer real dispararia, conferindo no fim que o estado continuava em COLLECTING. Precisei observar que o start_collection não cancela o timer de ativação, então quem impede a hot window de ativar é a checagem de COLLECTING dentro do _activate. Por isso disparar o callback manualmente depois de iniciar a coleta continua testando exatamente esse comportamento.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Unnecessary Sleep: removi os dois time.sleep e substituí o threading.Timer por um timer falso via monkeypatch, disparando o callback de ativação logo depois de iniciar a coleta. – O assert original foi mantido: o estado deve continuar COLLECTING mesmo depois do disparo do timer.

Refatoração 25

branch: refactor/sleepy-test
commit (dc8f813): refactor: fire hot window expiry timer directly

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_state_manager.py **Linha:** 179 e 183 (test_hot_window_expires_after_duration)

Minhas principais dificuldades na remoção do code smell são:
Dificuldade média. O teste dormia 40ms esperando a ativação da hot window e depois mais 100ms esperando a expiração, dependendo de dois threading.Timer reais. Precisei observar que o timer de expiração só é criado dentro do callback de ativação, então a ordem importa: o primeiro timer falso registrado é o de ativação e o segundo é o de expiração. Também tive que conferir que o _expire valida a geração e o estado HOT_WINDOW, por isso ele só pode ser disparado depois da ativação.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Unnecessary Sleep: removi os dois time.sleep e substituí o threading.Timer por um timer falso via monkeypatch, disparando primeiro o callback de ativação e depois o de expiração. – Os asserts originais foram mantidos: a hot window fica ativa após a ativação e volta para WAKE_WORD após a expiração.

Refatoração 26

branch: refactor/sleepy-test
commit (6d2f86f): refactor: activate hot window before manual expiry without sleeping

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_state_manager.py **Linha:** 195 (test_manual_expire_hot_window)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O teste dava um time.sleep(0.04) só para esperar o timer real ativar a hot window antes de expirá-la manualmente com expire_hot_window(). O sleep não fazia parte do comportamento testado, era só preparação. Precisei apenas garantir que a hot window estivesse realmente ativa antes da expiração manual, senão o teste passaria sem testar nada.

Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Unnecessary Sleep: removi o time.sleep e substituí o threading.Timer por um timer falso via monkeypatch, disparando o callback de ativação diretamente. – Add Assertion: além do assert de que a hot window fica inativa, adicionei um assert de que o estado volta para WAKE_WORD após a expiração manual.

Refatoração 27

branch: refactor/sleepy-test
commit (ec63ac8): refactor: advance hot window expiry check clock directly

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_state_manager.py **Linha:** 279 (test_check_hot_window_expiry_fallback)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O teste colocava a hot window ativa manualmente, conferia que ainda não tinha expirado e dava um time.sleep(0.06) para passar dos 50ms antes de chamar o check_hot_window_expiry() de novo. Diferente das refatorações anteriores, aqui não há timer envolvido: a checagem síncrona só compara time.time() com o início da janela. Precisei observar que o _hot_window_start_time tinha que ser definido com o mesmo relógio falso, senão a diferença de tempo ficaria errada.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell:

– Remove Unnecessary Sleep: troquei o time.sleep por um relógio controlado via monkeypatch no time do state_manager, avançando o tempo um pouco além de hot_window_seconds.
– Os asserts originais foram mantidos: não expira antes do tempo, expira depois e o estado volta para WAKE_WORD.

Refatoração 28

branch: refactor/sleepy-test
commit (b695c0f): refactor: drive post-expiry speech check with fake clock and timers

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Sleepy Test **Arquivo editado:** tests/test_state_manager.py **Linha:** 336 e 340 (test_speech_after_expiry_not_detected)
Minhas principais dificuldades na remoção do code smell são:
Dificuldade média. O teste dormia 40ms esperando a hot window ativar e depois mais 80ms esperando ela expirar, para então conferir que uma fala iniciada depois da expiração não era aceita. Foi a mais trabalhosa até agora porque precisei combinar as duas técnicas: o timer falso para disparar ativação e expiração, e o relógio falso porque o was_speech_during_hot_window compara o início da fala com o intervalo da janela, cujas marcas de início e fim são gravadas com time.time(). Tive que avançar o relógio na ordem certa (ativação, expiração e só depois a fala) para que o início da fala ficasse realmente depois do fim da janela.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Sleepy Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Unnecessary Sleep: removi os dois time.sleep, substituindo o threading.Timer por um timer falso e o time do state_manager por um relógio controlado, ambos via monkeypatch. – Os asserts originais foram mantidos: a hot window ativa, depois expira, e a fala iniciada após a expiração não é considerada parte da janela.

Refatoração 29

branch: refactor/unknown-test
commit (adc1b88): refactor: assert interrupted Piper TTS is not speaking

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Unknown Test **Arquivo editado:** tests/test_piper_tts.py **Linha:** 512 (test_multiple_interrupts_safe)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O teste chamava interrupt() dez vezes seguidas e só passava por não lançar exceção, sem nenhum assert. Precisei olhar o PiperTTS para achar um estado observável depois das interrupções e vi que o is_speaking() reflete se o motor está falando. Como nenhuma fala foi enfileirada, depois das interrupções ele deve continuar sem falar.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Unknown Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Add Assertion: adicionei um assert de que o TTS não está falando depois das múltiplas chamadas de interrupt(). – O comportamento original de não lançar exceção continua sendo verificado, agora junto com um resultado explícito.

Refatoração 30

branch: refactor/unknown-test
commit (87633ba): refactor: assert Piper TTS stays silent after start-stop cycles

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Tipo de test smell: Unknown Test **Arquivo editado:** tests/test_piper_tts.py **Linha:** 522 (test_start_stop_cycle)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O teste fazia três ciclos de start() e stop() com o TTS desabilitado e não tinha nenhum assert, passando só por não lançar exceção. Precisei observar que a única informação pública sobre o estado do motor é o is_speaking(), já que a thread de trabalho fica num atributo privado. Preferi assertar no comportamento público em vez de inspecionar o _thread, para não amarrar o teste à implementação.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Unknown Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Add Assertion: adicionei um assert de que o TTS não está falando depois dos ciclos de start() e stop(). – O comportamento original de os ciclos não lançarem exceção continua sendo verificado, agora com um resultado explícito.

Refatoração 31

branch: refactor/unknown-test
commit (34ee927): refactor: assert concurrent speaks disable unavailable Piper voice

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Unknown Test **Arquivo editado:** tests/test_piper_tts.py **Linha:** 533 (test_concurrent_speaks)
Minhas principais dificuldades na remoção do code smell são:
Dificuldade média. O teste disparava três threads chamando speak() dez vezes cada e terminava só com um comentário dizendo que a fila deveria ter itens, sem nenhum assert.

Antes de adicionar o assert eu rodei o cenário e vi que o comentário estava errado: como o modelo é falso, o speak() tenta iniciar o worker, o start() não consegue carregar a voz e desabilita o motor, então nada entra na fila. Precisei assertar o comportamento real e não o que o comentário prometia.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Unknown Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Add Assertion: adicionei asserts de que o motor fica desabilitado após as chamadas concorrentes e de que ele não está falando. – Também corrigi o comentário enganoso, que dizia que a fila teria itens, para descrever o que o teste realmente verifica.

Refatoração 32

branch: refactor/unknown-test
commit (84817aa): refactor: assert empty clipboard paste touches nothing

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Unknown Test **Arquivo editado:** tests/test_dictation.py **Linha:** 631 (test_clipboard_paste_empty_string_is_noop)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O teste chamava _clipboard_paste("") e _clipboard_paste(None) e só passava por não lançar exceção, sem nenhum assert. Precisei observar o que um "no-op" significa nesse caso: a função não pode escrever na área de transferência de nenhum sistema operacional nem simular o atalho de colar. Para verificar isso, segui o mesmo estilo dos testes vizinhos da classe TestClipboard, usando @patch nas funções de clipboard e no pynput_keyboard.
Usei o CATSPY para:

Usei o CATSPY para consultar a definição de Unknown Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Add Assertion: adicionei asserts de que nenhuma das funções de clipboard (Windows, macOS e Linux) foi chamada e de que nenhum controlador de teclado foi criado. – O comportamento original de não lançar exceção com texto vazio ou None continua sendo verificado.

Refatoração 33

branch: refactor/unknown-test
commit (95e8cc1): refactor: assert dictation transcribes without processing callback

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Unknown Test **Arquivo editado:** tests/test_dictation.py **Linha:** 923 (test_processing_callback_optional)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O teste criava o engine sem o callback on_dictation_processing_start, chamava o _finalise_and_transcribe e só tinha um comentário dizendo que não deveria lançar exceção. Precisei ver no código o que significa "o engine funcionar" nesse caso: depois de fechar o stream e tocar o beep, o fluxo tem que seguir para a transcrição mesmo sem o callback. O _transcribe_and_paste já estava mockado no teste, então bastou capturar esse mock e verificar a chamada.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Unknown Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell:

– Add Assertion: adicionei um assert de que o _transcribe_and_paste foi chamado uma vez com os mesmos frames de áudio recebidos, comprovando que a ausência do callback não interrompe o fluxo. – Extraí os frames de áudio para uma variável para poder comparar o argumento repassado à transcrição.

Refatoração 34

branch: refactor/unknown-test
commit (eb831e7): refactor: assert malformed chat IPC line leaves transcript empty

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Unknown Test **Arquivo editado:** tests/test_chat_window.py **Linha:** 395 (test_dispatch_malformed_line_is_swallowed)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade no código, mas tive um problema de ambiente. O teste enviava uma linha IPC de chat com JSON inválido para o _on_chat_ipc_line e só tinha um comentário dizendo que não deveria lançar exceção, que a janela seria criada sob demanda e que nenhum texto de resposta apareceria. O próprio comentário já indicava o que assertar. A dificuldade foi que o arquivo inteiro falhava no meu ambiente por uma incompatibilidade entre o mcp e a versão nova do pydantic, que eu precisei corrigir antes de conseguir validar a refatoração.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Unknown Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Add Assertion: adicionei um assert de que a janela de chat foi criada e outro de que o transcript continua vazio depois da linha malformada. – Os asserts transformam em verificação explícita o que antes só estava descrito no comentário do teste.

Refatoração 35

branch: refactor/conditional-test-logic
commit (08392d9): refactor: supply query and tool embeddings without branches

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Conditional Test Logic **Arquivo editado:** tests/test_tool_selection.py **Linha:** 292 (test_returns_minimum_tools)
Minhas principais dificuldades na remoção do code smell são:
Dificuldade média. O teste usava uma função falsa de embedding com um contador e um if para devolver um vetor na primeira chamada (a da query) e outro vetor em todas as chamadas seguintes (as das ferramentas), deixando o resultado depender de uma condição dentro do teste. Precisei olhar o _select_embedding para confirmar que a query é sempre embedada antes do loop das ferramentas, e que o número de ferramentas pode variar, então não dava para usar uma lista fixa de retornos sem acoplar o teste à quantidade de ferramentas built-in.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Conditional Test Logic e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Replace Conditional with Fixed Sequence: troquei a função com contador e if por um side_effect construído com chain([query_vec], repeat(tool_vec)), que entrega o vetor da query primeiro e repete o vetor das ferramentas depois, sem nenhum desvio. – O assert original foi mantido: mesmo com similaridade baixa, o resultado tem pelo menos 3 ferramentas.

Refatoração 36

branch: refactor/conditional-test-logic
commit (3f71ddd): refactor: parametrize expected outcome per file operation

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Tipo de test smell: Conditional Test Logic **Arquivo editado:** tests/test_local_files_path_arguments.py **Linha:** 37 (test_string_paths_keep_file_operations)
Minhas principais dificuldades na remoção do code smell são:
Dificuldade média. O teste já era parametrizado por operação e por caminho relativo ou absoluto, mas dentro dele uma cadeia de if/elif/else escolhia qual assert rodar conforme a operação, e dois ternários if relative else escolhiam o nome do arquivo e o formato do caminho. Precisei observar que cada combinação tinha que continuar verificando exatamente o mesmo resultado de antes (leitura devolve o conteúdo, escrita substitui, append concatena, delete remove e list mostra o nome), e manter os 10 casos gerados pelo parametrize.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Conditional Test Logic e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Parametrize Test: movi a expectativa de cada operação para o parametrize, junto com a operação, e o teste passou a ter um único assert direto sem if/elif. – Também troquei os ternários de caminho relativo/absoluto por parâmetros com o nome do arquivo e a forma de montar o caminho, usando id's legíveis para cada caso.

Refatoração 37

branch: refactor/conditional-test-logic
commit (144f828): refactor: stream fetch fixture page as a single chunk

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Conditional Test Logic **Arquivo editado:** tests/test_tools.py **Linha:** 206 (MockResponse.iter_content em test_fetch_web_page_success)

Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O dublê de resposta HTTP tinha um for no iter_content que fatiava o HTML de teste em pedaços do tamanho de chunk_size, colocando um loop dentro do teste. Precisei olhar o fetch_web_page.py para confirmar que a ferramenta só junta os chunks num buffer e confere o limite total de download. Como a página de teste tem poucas centenas de bytes e o chunk de leitura é de 64 KiB, o loop sempre gerava um único pedaço, então entregar o conteúdo inteiro de uma vez não muda o comportamento.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Conditional Test Logic e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Remove Loop from Test Double: troquei o for com yield por return iter([self.content]), deixando o dublê sem nenhuma lógica de repetição. – Os asserts originais foram mantidos: a ferramenta retorna sucesso e a resposta contém a URL da página.

Refatoração 38

branch: refactor/verbose-test
commit (3abe309): refactor: focus tool-call filtering test on stored chunks

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Verbose Test **Arquivo editado:** tests/test_dialogue_memory.py **Linhas:** 134 a 197 (test_dialogue_memory_filters_tool_calls)
Minhas principais dificuldades na remoção do code smell são:
Dificuldade média. O teste tinha mais de 60 linhas: doze linhas configurando atributo por atributo um Mock de config, comentários que só repetiam o código ("Mock database and config", "Run reply engine"), uma variável result que nunca era usada e quatro verificações separadas sobre os chunks (tamanho, dois in e um for procurando o id da tool call). Precisei observar que comparar a lista exata de chunks cobre tudo isso de uma

vez: se a tool call fosse guardada, a lista teria outro tamanho ou conteúdo e o assert falharia. Também conferi que get_pending_chunks() devolve os chunks na ordem em que foram adicionados, para a comparação de lista ser estável.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Verbose Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Extract Method: movi a montagem do Mock de configuração para o helper _reply_cfg() da classe, criado com argumentos nomeados em vez de doze atribuições. – Remove Redundant Code / Merge Assertions: removi comentários óbvios e a variável result sem uso, e troquei as quatro verificações sobre os chunks por um único assert comparando a lista exata (mensagem do usuário e resposta final), que também garante que a tool call não foi guardada.

Refatoração 39

branch: refactor/verbose-test
commit (139ab48): refactor: extract layout widget helper in hidden-window test

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Verbose Test **Arquivo editado:** tests/test_dictation_history.py **Linhas:** 280 a 330 (test_on_new_entry_is_safe_when_window_hidden)
Minhas principais dificuldades na remoção do code smell são:
Dificuldade média. O teste tinha cerca de 50 linhas: um docstring de dez linhas contando o histórico do crash no Windows, três trechos repetidos percorrendo o layout com itemAt(i).widget() (antes do sinal, depois do sinal e após o reload) e um for com if só para montar a lista de textos renderizados. Precisei observar que o layout pode ter itens sem widget (espaçadores), então o helper tinha que tolerar widget() devolvendo None sem quebrar a busca pelo texto, e que o teste precisava continuar verificando as duas coisas: nada muda com a janela escondida e a entrada aparece depois do reload.
Usei o CATSPY para:

Usei o CATSPY para consultar a definição de Verbose Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Eu estou usando os seguintes métodos de refatoração para remover o code smell: – Extract Method: criei o helper _layout_widgets(window) na classe e substituí as três varreduras repetidas do layout por chamadas a ele. – Remove Redundant Code: resumi o docstring ao comportamento testado, removi comentários que repetiam o código e troquei o for com if por uma list comprehension com getattr(w, "_entry", {}).get("text"), mantendo os dois asserts originais.

Refatoração 40

branch: refactor/verbose-test
commit (d74e84f): refactor: trim topic-welding prompt test to its assertions

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Tipo de test smell: Verbose Test **Arquivo editado:** tests/test_diary_poisoning_defence.py **Linhas:** 136 a 183 (test_prompt_forbids_welding_unrelated_topics)
Minhas principais dificuldades na remoção do code smell são:
Sem dificuldade. O teste tinha cerca de 48 linhas, mas só três asserts sobre o prompt do sumarizador. A maior parte era um docstring de dezessete linhas contando todo o incidente do Possessor/Jarvis e comentários antes de cada assert que repetiam o que a própria mensagem do assert já dizia. Precisei observar que nenhuma das três verificações podia ser alterada (proibir juntar tópicos, citar o mecanismo do aposto e trazer o exemplo Possessor/Jarvis), porque elas protegem o contrato do prompt descrito no summariser.spec.md.
Usei o CATSPY para:
Usei o CATSPY para consultar a definição de Verbose Test e ver uma possível refatoração
Eu estou usando os seguintes métodos de refatoração para remover o code smell:



UNIVERSIDADE
FEDERAL DO CEARÁ

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
– Remove Redundant Code: resumi o docstring ao comportamento verificado e removi os comentários que duplicavam as mensagens dos asserts.
– Inline Temp: juntei a variável prompt, que só era usada para gerar lowered, numa única linha. Os três asserts e suas mensagens foram mantidos sem alteração.